Table of Contents

Course Objective....2
Course Content....2
Hibernate object states....3
Storing and loading objects....7
Making an Object persistent (CREATE)....8
Retrieving a persistent object (READ)....9
Modifying a persistent object (UPDATE)....10
Making a persistent object transient (DELETE)....11
Making a detached object transient (DELETE)....12
Working with detached objects....13
Understanding equals() and hashCode()....14
Summary....14

Course Objective

1. Hibernate object states
2. Creating a new object and making it persistent (CREATE)
3. Loading an object (READ)
4. Modifying persistent and detached objects (UPDATE)
5. Deleting persistent and detached objects (DELETE)
6. Java identity/object equality/database identity

Course Content

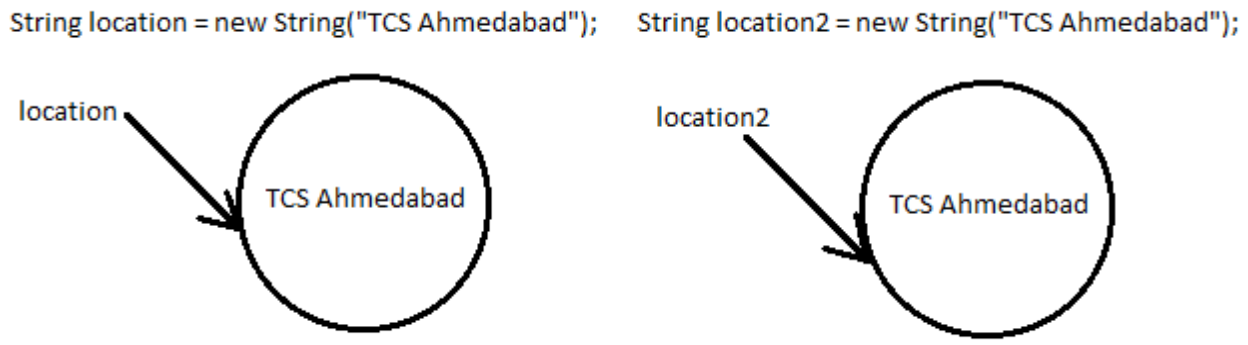
Java Identity & Object equality:

Java developers understand the difference between Java object identity and equality.

Object identity , ==, is a notion defined by the JVM machine. Two objects references are identical if they point to the same memory location.

On the other hand, object equality is a notion defined by classes that implement the equals() method. It means that two different (non identical) objects have the same value.

Example: Two different instances of String are equal if they represent the same sequence of characters, even though they each have their own location in the memory space of JVM.



Persistence complicates this picture. With object/relational persistence, a persistent object is an in-memory representation of a particular row of a database table. Along with Java identity(memory

location) and object equality, you pickup database identity (which is the location in the database).

Hibernate object states

Hibernate defines and supports the following object states:

You will see the case study of Book, Customer and BookTransaction being taken to discuss the concept of object states. Assume the following DB and Hibernate design.

VILP_TBL_BOOKS			
ID	**TITLE**	**CATEGORY**	**AUTHOR**
1	2 States	Fiction	Chetan Bhagat
2	The Power	Rhonda Byrne	Motivation
3	Data Structure	Technical	John Daniel
4	Revolution 2020	Fiction	Chetan Bhagat

VILP_CUSTOMERS			
CUSTOMER ID	**NAME**	**EMAIL**	**ADDRESS**
1	AJAY	AJAY@VILP.COM	STUDIO, INFOCITY, GANDHINAGAR
2	ROHIT	ROHIT@VILP.COM	MADHURAM, GANDHINAGAR
3	GOPAL	GOPAL@VILP.COM	MADHURAM, GANDHINAGAR
4	AMIT	AMIT@VILP.COM	STUDIO, INFOCITY, GANDHINAGAR

VILP_BOOK_TRANSACTION				
TRANSACTION ID	**CUSTOMER ID**	**BOOK ID**	**ISSUED DATE**	**DUE DATE**
1	1	4	1. May. 2014	15. May. 2014
2	2	2	1. May. 2014	15. May. 2014
3	3	2	3. May. 2014	18. May. 2014
4	2	1	7. May. 2014	21. May. 2014
5	1	2	11. May. 2014	26. May. 2014
6	2	3	15. May. 2014	30. May. 2014
7	3	3	15. May. 2014	30. May. 2014

We will create following POJO to represent relationship in Java classes.

```
public class BookTransactionDto
{
        private int transactionId;
        private Date issueDate;
        private Date dueDate;

        private Customer customerDto;

        private Book bookDto;

        ...
        ...
}
```

```
public class Book
{
        private int bookId;
        private String title;
        private String author;
        ...
        ...
}

public class Customer
{
        private int id;
        private String name;
        private String email;
        private String address;
        ...
        ...
}
```

Transient -

1. An object is transient if it has just been instantiated using the new operator, and it is not associated with a Hibernate Session.
2. It has no persistent representation in the database and no identifier value has been assigned.
3. Transient instances will be destroyed by the garbage collector if the application does not hold a reference anymore.
4. Use the Hibernate Session to make an object ***persistent*** (and let Hibernate take care of the SQL statements that need to be executed for this transition).

The below code represents the save or add operation. Assume that the code, that is striked off, is not written.

```
~~Session session = factory.openSession();~~
~~session.beginTransaction();~~

Book book = new Book("2 States", "C. Bhagat", "Fiction");
~~session.save(b);~~
~~session.getTransaction().commit();~~
~~session.close();~~
```

Few points to note about the code that is not striked off.

1. The ***book*** object has just been instantiated using the new operator. It is not associated with a hibernate session.
2. It has not been saved to database yet. The value for ***bookId*** is 0.
3. Hence the book object is transient.
4. It could have been made persistent (saved to DB) by called ***session.save()***.

Persistent -

1. A ***persistent*** instance has a representation in the database and an identifier value.
2. It might just have been saved or loaded, however, it is by definition in the scope of a Session.
3. Hibernate will detect any changes made to an object in persistent state and will automatically synchronize the state with the database when the unit of work completes. ie when the Session closes.
4. Developers do not need to execute manual UPDATE statements. Also no need to execute DELETE statements when an object should be made transient.

```
Session session = factory.openSession();
session.beginTransaction();

Book book = new Book("2 States", "C. Bhagat", "Fiction");
session.save(b);

session.getTransaction().commit();
session.close();
```

Few points to note about the code that is not striked off.

1. The reference ***book*** now has a representation (row) in the database. Hibernate has also assigned the primary key value by calling it's setBookId() method. A call to book.getBookId() will return the primary key generated when ***book*** reference in saved to DB.
2. The reference ***book*** is said to be ***persistent*** only in the scope of session. Once the session is closed, the reference ***book*** will no longer be called persistent.
3. If the reference ***book*** changes its state (change in value of any of it's attribute), hibernate will detect these changes and synchronize the state with the database.

Detached -

1. A ***detached*** instance is an object that has been persistent, but its Session has been closed.

2. The reference to the object is still valid, of course, and the ***detached*** instance might even be modified in this state.
3. A ***detached*** instance can be reattached to a new Session at a later point in time, making it (and all the modifications) persistent again.
4. This feature enables a programming model for long running units of work that require user think-time. We call them application transactions, i.e., a unit of work from the point of view of the user.

```
Session session = factory.openSession();
session.beginTransaction();

Book book = new Book("2 States","C. Bhagat","Fiction");

session.save(book); ← 1

session.getTransaction().commit();
session.close(); ← 2
```

Few points to note about the code:

1. At block 1, the reference ***book*** was persistent, but as the Session is closed at block 2, the reference book after line 2 will be considered as detached.
2. In the scope of Java, the reference ***book*** is still valid and you can even modify the state of reference ***book***.
3. The reference ***book*** can be reattached to a new Session at later point of time.

Storing and loading objects

We will now discuss the states and state transitions (and the Hibernate methods that trigger a transition) in more detail.

Any persistence tool (like Hibernate etc.) includes a persistence manager API. The persistence manager may have several different interfaces. In case hibernate, these are ***Session***, ***Query***, ***Criteria*** and ***Transaction***.

We will now see how to load and store objects with Hibernate.

In a Hibernate application, you store and load objects by essentially changing their state. You do this in units of work. A single unit of work is a set of operations considered as an atomic group. A unit of work is a particular sequence of state changes to your objects that you'd group together.

<u>Beginning a unit of work</u>

At the beginning of a unit of work, an application obtains an instance of ***Session*** from the application's ***SessionFactory***:

Session session = SessionFactory.openSession();

At this point a new persistence context is also initialized for you, and it will manage all the objects you work with in that ***Session***.

The application may have multiple Session Factories if it accesses several databases. You should never create a new SessionFactory just to service a particular request. Creation of a SessionFactory is expensive. On the other hand, Session creation is extremely inexpensive. The Session does not even obtain a JDBC Connection until a connection is required.

An unit of work must occur inside a transaction, no matter if you read or write data. The below code begins a Transaction.

Transaction tx = session.beginTransaction();

Making an Object persistent (CREATE)

With a Session, you can make a new transient object persistent with the save() method.

```
Book book = new Book();
book.setAuthor("Khushwant Singh");
book.setTitle("Train to Pakistan");
book.setCategory("Fiction");
```
← **1**

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
```
← **2**

```
session.save(book);
```
← **3**

```
tx.commit();
```
← **4**

```
session.close();
```
← **5**

1. A new **transient** object Book(shown in block 1 above) is instantiated as usual. Of course, you may also instantiate it after opening a Session; they aren't related yet.
2. A new Session is opened using the SessionFactory. You start a new Transaction(block 2).
3. A call to save() (block 3) make the transient instance of Book **persistent**. It is now associated with the current Session.
4. On commit (block 4)Hibernate synchronizes the persistence context. We say a flush (can be called manually by flush()) occurs.
5. Session can finally be closed, and the persistence context ends. The reference *book* is now a reference to an object in **detached** state.

Retrieving a persistent object (READ)

The Session is also used to query the database and retrieve existing persistent objects. Two special methods are provided for the simplest kind of query: retrieval by identifier: the get() and load() methods.

The retrieved object ***book*** is in persistent state and as soon as the persistence context is closed, the ***book*** object goes to detached state.

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Book book = (Book)session.get(Book.class, 22);

//Book book = (Book)session.load(Book.class, 22);

tx.commit();
session.close();
```

The one difference between get() and load() is how they indicate that the instance could not be found. If no row with the given identifier value exists in the database, get() return null. The load() method throws an ObjectNotFoundException().

More important, the load() method may return a proxy, without hitting the database. A consequence of this is that you may get an ObjectNotFoundException later, as soon as you try to access the returned proxy(i.e. accessing methods and attributes of the object, in this case Book object).

The load() method always tries to return a proxy, and only returns an initialized object instance. In the example shown earlier, no database hit occurs at all. The get() method on the other hand never

returns a proxy, it always hits the database.

Purpose of load():

It is common to obtain a persistent instance to assign it as a reference to another instance. For example, imagine that you need the book only for a single purpose: to set an association with a BookTransaction. If this is all you plan to do with the book, a proxy will do fine, there is no need to hit the database. In other words, when the BookTransaction is saved, you need the foreign key value of a book inserted into the BOOK table. The proxy of a Book provides just that: an identifier value wrapper in a proxy that looks like the real thing.

Modifying a persistent object (UPDATE)

Any persistent object returned by get(), load(), or any entity queried is already associated with the current Session and persistence context. It can be modified, and its state is automatically synchronized with the database.

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Book book = (Book)session.get(Book.class, 22);  <- 1

book.setCategory("Fiction2");  <- 2

tx.commit();  <- 3

session.close();  <- 4
```

1. You retrieve the object from the database with the given identifier.
2. You modify the object.
3. These modifications are propagated to the database when ***tx.commit()*** is called. This mechanism is called automatic dirty checking.
4. As soon as you close the Session, the instance ***book*** is considered detached.

Making a persistent object transient (DELETE)

You can easily make a persistent object transient, removing its persistent state from the database, with the delete() method.

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Book book = (Book)session.load(Book.class, 22);  ← 1

session.delete(book);  ← 2

tx.commit();  ← 3

session.close();  ← 4
```

1. An object has to be loaded into the persistence context; an instance has to be in persistent state to be removed. A proxy is good enough (not necessary to call get()).
2. The ***book*** object is in removed state after you call delete(); you shouldn't continue working with it. You should make sure any reference to it in your application is removed.
3. The SQL DELETE is executed only when the Session's persistence context is synchronized with the database at the end of the unit of work.
4. After the session is closed, the ***book*** object is considered an ordinary transient instance. The transient instance is destroyed by the garbage collector if it's no longer referenced by any

other object. Both the in-memory object instance and the persistent database row will have been removed.

Making a detached object transient (DELETE)

You can make a detached instance transient, deleting its persistent state from the database. You don't have to reattach a detached instance to delete it from the database.

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Book book = (Book)session.get(Book.class, 22);   <- 1

tx.commit();

session.close();   <- 2

Session session2 = factory.openSession();
Transaction tx2 = session2.beginTransaction();   <- 3

session.delete(book);   <- 4

tx2.commit();   <- 5

session2.close();
```

1. You retrieve the object from the database with the given identifier.
2. Session is closed, and the persistence context ends. The reference ***book*** is now a reference to an object in detached state.
3. A new Session is opened using the SessionFactory. You start a new Transaction.
4. The call to delete() does two things: It reattaches the object to the Session and then schedules the object for deletion, executed on commit(). You shouldn't continue working with it. You should make sure any reference to it in your application is removed.
5. The SQL DELETE is executed only when the Session's persistence context is synchronized with the database at the end of the unit of work. After the session is closed, the ***book*** object is considered an ordinary transient instance.

Working with detached objects

Modifying the ***book*** after the Session is closed has no effect on its persistent representation in the database. As soon as the Session is closed, book becomes a detached instance. If you want to save modifications you made to a detached object, you have to either ***reattach*** or ***merge*** it.

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Book book = (Book)session.get(Book.class, 22);   ← 1

tx.commit();

session.close();   ← 2

book.setAuthor("Khushwant Singh");   ← 3

Session session2 = factory.openSession();
Transaction tx2 = session2.beginTransaction();   ← 4

session2.update(book);   ← 5

book.setCategory("Fiction2");   ← 6

tx2.commit();   ← 7

session2.close();
```

1. You retrieve the object from the database with the given identifier.
2. Session is closed, and the persistence context ends. The reference ***book*** is now a reference to an object in detached state.
3. The detached object ***book*** is modified.
4. A new Session is opened using the SessionFactory. You start a new Transaction.
5. A detached instance may be reattached to a new Session by calling update() on the detached object.
6. The persistent ***object*** book is modified. It does not matter if the ***book*** object is modified before or after it is passed to update().
7. Hibernate always treats the object as dirty and schedules an SQL update, which will be executed during flush/commit.

Understanding equals() and hashCode()

Many Java developers never had to override the equals() and hashCode() methods before using Hibernate. They seem to be unaware of the intricate details of such an implementation. Every object-oriented programming language with hash-based collections requires a custom equality routine. The detached object state in a Hibernate application exposes you to this problem, may be for the first

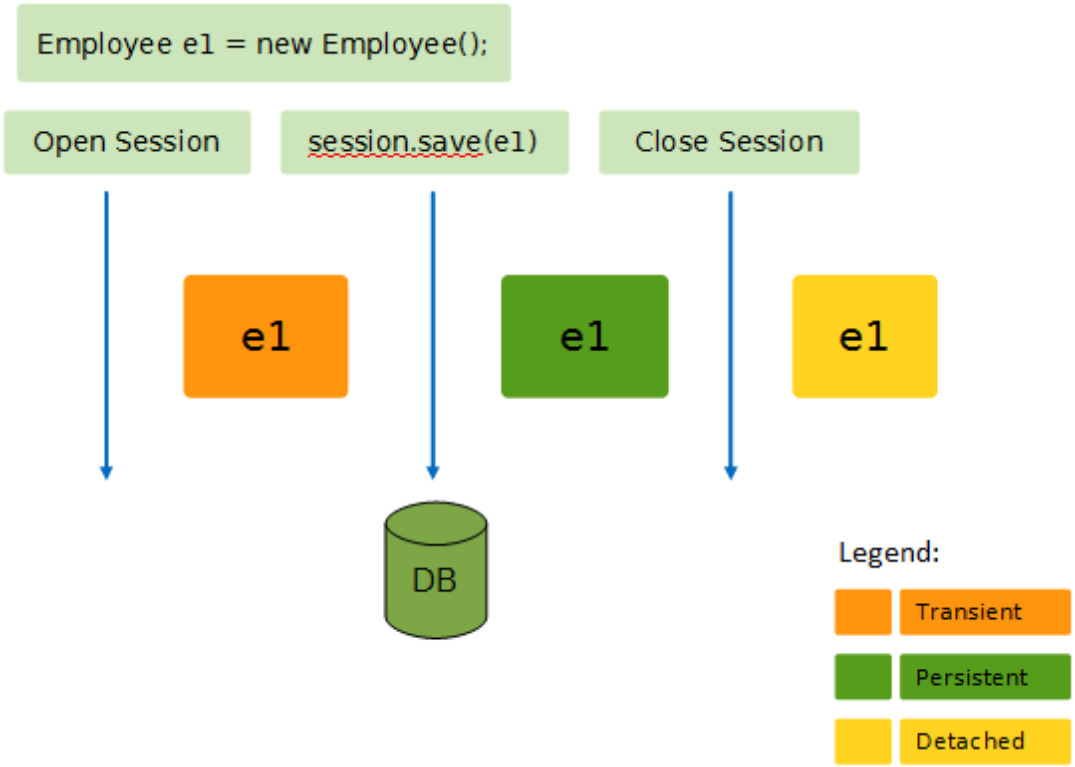
time.

In your application, you have detached objects and that you have to test them for equality (because you want to add them to Set collection, etc.) with your own logic. You must implement equals() and hashCode(). Keep in mind that when you override equals(), you always need to override hashCode() so the two methods are consistent.

Scope of object identity

The hibernate identity (obj1 == obj2) scope is the scope of a persistence context (Session). If you request two objects using the same database identifier value in the same Session, the result is two references to the same in-memory instance.

Summary



You now know, that:

1. A transient instance can become persistent instance by calling save() on transient.
2. A persistent instance becomes a detached instance after closing the session.
3. A persistent instance or a detached instance can become transient instance by calling delete() method.